

CPU Scheduling Simulator

Revised Student Project Report

Operating Systems I • Summer 2026 • Team 8 • Variant 3 (FCFS, SRTF, Priority Preemptive)

Revision note: Section 1 has been updated to separate Team Members into a standalone table and present Tasks in a separate dedicated table beneath it.

1. Team Members and Tasks

Team Members

Student ID	Student Name
20220473	Mariam Hany Refaat
20230241	Zainab Abdelfattah Darwish
20240929	Mariam Ahmed Mohammed
20240768	Mazen Mohamed Sobhy
20210291	Hassan Abdelrahman Hassan
20240256	Jana Hany Mahmoud Tawfiq

Project Tasks

Tasks
Process Model & Input subsystem
Algo 1 & Algo 2
Algo 3
Simulation timeline & metrics & text gantt chart
Integration & testing
Documentation

2. Project Overview

The project is a C command-line simulator for CPU scheduling. It accepts a workload through manual or file input, validates process data, runs the three algorithms assigned to Variant 3, records an execution timeline, generates a text Gantt chart, calculates scheduling metrics, and compares the algorithms on the same original workload.

Variant 3 uses FCFS as the common non-preemptive baseline, SRTF as a preemptive shortest-remaining-time policy, and Priority Preemptive as a preemptive policy where a smaller numerical priority value is better. The simulator is a model and does not modify the Linux scheduler.

3. Project Architecture

Module	Responsibility	Main functions / data
process.h / process.c	Process representation, initialization, reset and validation	Process; process_init, process_reset_runtime; process_validate
input.h / input.c	Manual / file input and validation	input_manual; input_from_file; input_print_workload
scheduler.h	Common scheduler API	fcfs_schedule; srtf_schedule; priority_schedule
fcfs.c	Non-preemptive FCFS	Arrival/PID ordering; idle segments
srtf.c	Preemptive SRTF	One-unit simulation; remaining-time selection
priority.c	Preemptive Priority	One-unit simulation; priority/arrival/PID selection
timeline.c / timeline.h	Shared execution timeline	Timeline Segment; timeline_add: IDLE=-1; merge
metrics.c	Per-process and average metrics	CT / TAT / WT / RT calculations
display.c / main.c	Gantt, menu, cached simulations and comparison	printGanttChart; run_and_save; run_all_simulations

Flow: Input & validation → Process model → FCFS / SRTF / Priority → Timeline → Metrics + Gantt Comparison.

4. Process Model and Input Subsystem

The shared Process structure separates original input data from runtime scheduling state and calculated statistics. Original fields are PID, arrival time, burst time and priority. Runtime fields include remaining_time, started and completed. Statistics include first_start_time, completion_time, turnaround_time, waiting_time and response_time.

Each scheduler uses a private copy of the original workload and resets runtime fields before simulation. This keeps comparison mode safe because one algorithm does not modify the process state used by another.

Manual input reads the process count and then PID, arrival time, burst time and priority. File input uses a first-line process count followed by PID Arrival Time Burst Time Priority records. The implementation supports up to 100 processes.

Validation rejects empty workloads, duplicate PIDs, negative arrival times and non-positive burst times. Invalid counts, malformed records, unreadable files and non-numeric data are handled with an error message.

5. Scheduling Algorithms

Algorithm	Selection rule	Preemption	Tie rule	Implementation
FCFS	Earliest arrival; equal arrivals by PID	None	Arrival then PID	src/fcfs.c / fcfs_schedule
SRTF	Smallest remaining time among ready processes	Yes; reconsidered each time unit	Remaining time, arrival, PID	src/srtf.c / srtf_schedule
Priority Preemptive	Smallest numerical priority	Yes; reconsidered each time unit	Priority, arrival, PID	src/priority.c / priority_schedule

5.1 FCFS

FCFS runs processes in arrival order and never interrupts a running process. Equal arrivals are resolved by smaller PID. CPU gaps are recorded as IDLE segments.

5.2 SRTF

SRTF advances one time unit at a time, scans ready unfinished processes and selects the smallest remaining time. A newly arrived process with a strictly shorter remaining time preempts the current process.

5.3 Priority Preemptive

Priority Preemptive uses the same one-unit simulation pattern, but selects the smallest numerical priority. Priority 1 is higher than priority 2. A newly arrived process preempts when its priority is strictly better.

6. Simulation Timeline, Metrics and Gantt Chart

The shared timeline stores start time, end time and PID for each execution segment. *PID* = -1 represents CPU idle time. Adjacent segments belonging to the same process are merged, while preemptions and idle periods remain visible.

Required metrics are derived from the timeline:

$$TAT = CT - AT \quad | \quad WT = TAT - BT \quad | \quad RT = First\ CPU\ Start - AT$$

Average waiting, turnaround and response times are calculated over the process metrics. The Gantt renderer reads the same timeline and prints the execution sequence with time boundaries.

7. Comparison Results

Representative workload: P1=(AT 0, BT 8, priority 2), P2=(AT 1, BT 4, priority 1), P3=(AT 2, BT 2, priority 3), P4=(AT 3, BT 1, priority 2).

Algorithm	Avg WT	Avg TAT	Avg RT
FCFS	7.00	10.75	7.00
SRTF	2.75	6.50	0.25
Priority Preemptive	6.00	9.75	5.00

Reference Gantt charts:

```
FCFS:          |0 -- P1 -- 8|8 -- P2 -- 12|12 -- P3 -- 14|14 -- P4 -- 15|
SRTF:          |0 - P1 - 1|1 - P2 - 2|2 - P3 - 4|4 - P4 - 5|5 - P2 - 8|8 - P1 - 15|
Priority Preemptive:|0 - P1 - 1|1 - P2 - 5|5 - P1 - 12|12 - P4 - 13|13 - P3 - 15|
```

8. Testing and Verification

The documented test suite contains ten workloads covering normal operation, different arrivals, initial and intermediate idle periods, ties, immediate preemption, invalid input, a single process and a combined edge case. All ten tests are recorded as passing.

ID	Purpose	Expected / Observed Result
T01	All processes at time 0	FCFS P1-P2-P3; SRTF P3-P2-P1; Priority P2-P1-P3 — Pass
T02	Different arrivals	SRTF preempts on shorter arrival; FCFS keeps arrival order — Pass
T03	Initial CPU idle	Timeline begins with IDLE — Pass
T04	Idle gap	IDLE segment appears between processes — Pass
T05	Scheduling ties	Arrival / PID tie rules observed — Pass
T06	Immediate preemption	Priority preempts when better priority arrives — Pass
T07	Empty / malformed input	Rejected safely — Pass
T08	Single process	Correct timeline and metrics — Pass
T09	Invalid burst / duplicate PID	Rejected; simulation not attempted — Pass
T10	Combined edge case / sample	Reference averages reproduced — Pass

9. Program Design Decisions

The project uses a multi-file C design with shared headers and common timeline, metrics and display services. FCFS, SRTF and Priority reuse the same Process structure. SRTF and Priority use a ready-process scan rather than a specialized heap; this is straightforward for a teaching simulator. Private process copies keep later simulations independent.

10. Difficulties, Limitations and Known Issues

- The current menu uses descriptive labels for SRTF and Priority Preemptive; the handbook's generic labels are Algorithm 2 and Algorithm 3. Confirm whether exact wording is required.
- The metrics module contains per-process metric formatting, while the normal menu path documented in the supplied report primarily shows averages and Gantt output. Verify visible CT/TAT/WT/RT before final submission.
- The supplied data/sample_input.txt has a trailing "2.simulator" token; tests/test10.txt is the clean equivalent used for verification.
- Root-level duplicate C files are legacy/unintegrated paths; the documented build uses src/*.c.
- The handbook does not define a separate numerical priority range; the implementation follows the stated smaller-number-is-higher convention.

11. Conclusion

The project provides an integrated Variant 3 CPU Scheduling Simulator in C with manual/file input, validation, FCFS, SRTF and Priority Preemptive scheduling, a shared timeline, text Gantt charts, metrics and comparison mode. The documented reference workload reproduces the handbook's expected averages, and all ten supplied tests are recorded as passing.

Appendix A - Build and Run

```
# Compile from repository root:
gcc -Wall -Wextra -Iinclude src/*.c -o simulator

# Run:
./simulator

# Sample workload input format:
4
1 0 8 2
```

```
2 1 4 1
3 2 2 3
4 3 1 2
```

Appendix B - Test Suite Index

File	Coverage
tests/test01.txt	All processes arrive at time 0; ordering / ties
tests/test02.txt	Different arrivals: SRTF preemption
tests/test03.txt	CPU initially idle
tests/test04.txt	Idle interval between processes
tests/test05.txt	Scheduling ties
tests/test06.txt	Immediate preemption
tests/test07.txt	Empty / malformed input
tests/test08.txt	Single process
tests/test09.txt	Zero burst and duplicate PID
tests/test10.txt	Combined edge case and official sample workload

Appendix C - Final Submission Checklist

- Confirm the final team-member contribution table before submission.
- Use the clean sample input and verify the active `src/` and `include/` build paths.
- Verify visible per-process CT/TAT/WT/RT in the final program if the instructor checks the interface strictly.
- Ensure the final submission contains source code, README, tests, sample data, sample output and this report.